Namespaced Functions

Prior to PHP 5.3, all functions lived in global scope, and name collisions were common. The same was especially true of classes. To avoid the problem, unwieldy names were often used to prefix function and class names to make them specific to a library, filesystem path, or some other abstract method of organizing code. With namespaces, however, it is possible to group related functions together (along with classes, interfaces, and constants). If we take the example of colliding function names from the previous section, we can isolate the two functions in their own namespaces like so:

```
namespace {
    function collision(): void {
        print "Hello from root namespace." . PHP_EOL;
    }
}

namespace Other {
    function collision(): void {
        print "Hello from Other namespace." . PHP_EOL;
    }
}

namespace Other\SubOther {
    function collision(): void {
        print "Hello from Other\\Sub namespace." . PHP_EOL;
    }
}
```

In the first declaration – `namespace { ... }` – we are simply telling the PHP engine that the code doesn't use a namespace, so PHP uses root. We define a function `collision()`. Next, we enter a new namespace called `Other` and define another function called `collision()`. Finally, we enter a third namespace `Other\Sub`, and create another `collision()` function. We now have three namespaces in our code: root (or non-namespaced), `Other`, and `Other\Sub`. `Sub` is a sub-namespace of `Other`.

This example comes with a disclaimer. *It is NOT recommended to use multiple namespace blocks like this in a single file.* There are very few valid use cases for doing so and usually involve generated code (or writing books about the language). Best practice is to use a single namespace statement per file and organize namespaced source code